From Conditional Flow Matching to Diffusion

In class we studied variational autoencoders (VAEs), which encode data into a random hidden representation and decode it into samples, and conditional flow matching, which learns a time-dependent velocity from paths joining noise and data. This homework supplies the Gaussian path and velocity formulas needed for implementation. Problem 1 extends the linear (conditional) flow matching from class; Problems 2–4 introduce diffusion-style flow training and generation on the familiar two-moons (two interleaving arcs) and Modified National Institute of Standards and Technology (MNIST; handwritten-digit images) datasets.

Data, code, and submission

Use scikit-learn's `make_moons` for two-dimensional (2D) data and Torchvision's original `MNIST` dataset for 28-by-28 digits, as in the two standalone flow-matching demos in `experiments/generative/`. Use PyTorch's `nn.Module`, standard layers, autograd (automatic differentiation), and `torch.optim`; you do not need to implement backpropagation or convolutions from scratch. Keep the reference implementation notebook-like. See `homework/generative/README_data.md` for data sources, splits, and preprocessing.

Problem 1: More than one time per linear flow pair

Recall the *linear* conditional flow-matching demo from class, with forward $t \in [0, 1]$ running from independent $x_0 \sim \mathcal{N}(0, I)$ to $x_1 \sim p_{\text{data}}$:

$$x_t = (1 - t)x_0 + tx_1, \qquad u_t = x_1 - x_0.$$

Here I is the identity covariance matrix (of size 2×2 for two moons) and p_{data} is the real two-moons distribution; u_t is the target velocity along a fixed noise/data pair. This path runs from noise to data as its time variable increases. Modify the two-moons standalone demo; repeating the study on MNIST is optional.

1.1 Modify the two-moons `Flow` network and loss loop to compare three time designs per fixed pair (x_{0i}, x_{1i}): $K = 1$ independent uniform time (the demo baseline), $K = 4$ independent uniform times, and $K = 4$ midpoint-grid times $t_j = (j + \frac{1}{2})/4$. For $K > 1$, average squared velocity error over the four time points, pair batch, and coordinates before *one* optimizer step. Show the tensor shapes and retain the same model initialization, optimizer, shared training-pair minibatch sequence (the small batch used for each update) within each seed, sampler, and held-out data (not used for optimizer updates) across designs.

1.2 On one fixed held-out (x_0, x_1, t) set, plot validation velocity mean squared error (MSE: the average squared difference between predicted and target velocity coordinates) and all four training-loss trajectories. Generate each design from the same initial Gaussian draws with the same second-order Runge–Kutta (RK2) midpoint grid. RK2 is a numerical integrator that evaluates the learned velocity at the current state and a predicted midpoint of each step.

Report squared maximum mean discrepancy (MMD^2) for generated versus real points using a radial basis function (RBF, or Gaussian) kernel $k_h(x, y) = \exp[-\|x - y\|^2/(2h^2)]$. The bandwidth $h > 0$ is the distance scale over which points count as similar; choose it using only real validation data and then keep it fixed. For generated set G and real set R, define $\overline{k_h(A, B)}$ as the average kernel value over all ordered pairs $(a, b) \in A \times B$, including self-pairs when $A = B$. Then the empirical MMD^2 is

$$\widehat{\text{MMD}}_h^2(G, R) = \overline{k_h(G, G)} + \overline{k_h(R, R)} - 2\overline{k_h(G, R)}.$$

It compares within-set similarities with generated–real similarity. Also report the same MMD^2 for two independent equal-size real sets (a real-versus-real finite-sample reference, not a pass/fail threshold). Show a labeled scatter for each design and individual and summary results for at least two seeds.

1.3 In 200–300 words, distinguish conclusions from update-matched comparisons (equal optimizer-step counts) and example-matched comparisons (equal numbers of evaluated pair–time inputs). Discuss time-grid bias (overemphasis on the chosen fixed times), stochastic gradient variance (fluctuation from randomly sampled training pairs and times), solver error (numerical integration error), and sampling uncertainty (variation from finite seeds and generated samples). Explain why lower training loss does not establish better generated samples.

Guided background: a Gaussian path and its flow

In the linear (conditional) flow matching, increasing time t carries Gaussian noise to data. Here, increasing time $s \in [0,1]$ instead *corrupts* clean data into almost-Gaussian noise. Generation will run in the opposite direction, from $s = 1$ down to $s_{\min} = 10^{-3} \approx 0$. Let X_{data} be a training point or image, c its digit label when applicable, and $\varepsilon \sim \mathcal{N}(0, I)$ an independent Gaussian tensor of the same shape. Define

$$\begin{aligned} \beta(s) &= 0.1 + 19.9s, & B(s) &= 0.1s + 9.95s^2, \\ a(s) &= e^{-B(s)/2}, & \sigma(s) &= \sqrt{1 - a(s)^2}, \\ X_s &= a(s)X_{\text{data}} + \sigma(s)\varepsilon. \end{aligned} \tag{G}$$

Here β is the chosen noise-rate schedule, $B(s) = \int_0^s \beta(r)\, dr$ is its cumulative rate, $a(s)$ scales the retained data signal, and $\sigma(s)$ scales the injected noise. The last line is the *Gaussian corruption path*. During training, we know both X_{data} and the injected ε. At a fixed pair $(X_{\text{data}}, \varepsilon)$, the path has the supplied conditional velocity

$$u_s = a'(s)X_{\text{data}} + \sigma'(s)\varepsilon = -\frac{\beta(s)}{2}X_s + \frac{\beta(s)}{2\sigma(s)}\varepsilon. \tag{C}$$

This is analogous to the pair-conditioned velocity target in the linear flow matching, although its path and time direction differ. At a location $X_s = x$, many training pairs can be compatible with that location. Noise-MSE (mean squared error between predicted and injected noise) training of $\varepsilon_\theta(s, x, c)$ learns the average injected noise for those pairs. Replacing ε in (C) by the predictor gives the *learned flow field* used for generation:

$$v_\theta(s, x, c) = -\frac{\beta(s)}{2}\left[x - \frac{\varepsilon_\theta(s, x, c)}{\sigma(s)}\right]. \tag{V}$$

For the unconditional two-moons task in Problem 3, omit the label c from the predictor and the field. In the ideal population limit this averaged field transports the one-time distributions of the Gaussian path; it need not follow any particular fixed-pair trajectory. An ordinary differential equation (ODE) specifies how a state changes continuously with time; this deterministic generator is called the *probability-flow ODE* because its ideal field matches the Gaussian path's one-time distributions.

For each optimizer update, sample clean data, time $s \sim U[s_{\min}, 1]$ (uniformly over that interval), and fresh ε; form X_s with (G) and minimize $\|\varepsilon_\theta(s, X_s, c) - \varepsilon\|^2$ averaged over batch and coordinates or pixels. To generate, draw $x_0 \sim \mathcal{N}(0, I)$ at $s_0 = 1$ and numerically solve $dx/ds = v_\theta(s, x, c)$ at decreasing times. After the last ODE step, use the final denoising estimate

$$\widehat{x}_{\text{data}} = \frac{x_{s_{\min}} - \sigma(s_{\min})\varepsilon_\theta(s_{\min}, x_{s_{\min}}, c)}{a(s_{\min})}. \tag{D}$$

The standard-normal start is approximate because $a(1) > 0$; stopping at $s_{\min} > 0$ leaves some noise, and (D) is an approximate denoising operation, not an additional exact ODE step. Avoid $s = 0$ in (V) because $\sigma(0) = 0$.

Problem 2: Some Math for Guided diffusion

2.1 Using the above formulas, evaluate $B(s)$, $a(s)$, and $\sigma(s)$ at $s = s_{\min}, 0.5, 1$; report $a(1)$ to three significant figures. Check numerically that $a(s)^2 + \sigma(s)^2 = 1$. In one or two sentences, explain why the start from $\mathcal{N}(0, I)$ is approximate.

2.2 Write pseudocode for one noise-MSE training update using (G). State the shapes of clean data, s, ε, X_s, and the model output for both a two-moons batch and an MNIST batch. Explain why a fixed set of held-out $(X_{\text{data}}, s, \varepsilon)$ triples gives a useful validation curve, and why the network must not take the clean observation as an input.

2.3 Use the field (V), without deriving it, to write one explicit Euler update (a first-order numerical step using the field at the current state and time) from s_k to $s_{k+1} < s_k$. State the sign of $s_{k+1} - s_k$, the inputs and output shape of ε_θ, and the number of network evaluations for 50 Euler steps plus (D). Explain in words how a velocity for one fixed pair in (C) differs from the learned field in (V). A score (a log-density gradient) or stochastic differential equation (SDE, an evolution rule with random noise) derivation is not expected.

2.4 Using the estimate (D) with a chosen $s > 0$ in place of $s_{\min}$, substitute a known $(x_{\text{data}}, \varepsilon, s)$ from (G) and show that the *same oracle* ε (the injected noise known by construction, not a model prediction) reconstructs x_{data} exactly in algebra. Describe one tensor/finite-value test for this identity, one numerical risk near $s = 0$, and why passing the oracle check does not prove realistic generated samples.

Problem 3: Diffusion ODE on two moons

3.1 Generate 1,000 training, 1,000 independent validation, and 1,000 independent evaluation-only two-moons points with `make_moons(noise=0.15)` using recorded distinct seeds. State its Gaussian noise model (the noise distribution, scale, and coordinate dependence) and plot the training and validation sets. Use float32 (32-bit floating-point) tensors of shape $(N, 2)$; keep the validation points out of optimizer updates.

3.2 Adapt the three-hidden-layer PyTorch multilayer perceptron (MLP, a fully connected feed-forward network) with exponential linear unit (ELU) activations in the two-moons flow demo to output *noise* rather than velocity. An ELU maps $z > 0$ to z and $z \le 0$ to $e^z - 1$. Sample fresh s and ε during training, form X_s with (G), and minimize coordinate-mean noise MSE. Plot training loss by update and validation loss on a *fixed* set of held-out (x, s, ε) triples. Verify output shape, finite gradients, and finite losses (no undefined or infinite numerical values). Compare validation MSE with the zero-noise predictor (which always predicts $\widehat{\varepsilon} = 0$); raw noise MSE is not itself sample quality.

3.3 Implement decreasing-s integration using the supplied noise-to-velocity formula (V). Start from 1,000 fixed standard-normal draws, use a 50-step explicit Euler grid from 1 to $s_{\min}$, and apply your stated final denoise. Plot generated and held-out real points on the same axes, plus at least five saved trajectory times. Report runtime and number of network evaluations (forward passes through the denoising network). Check sampler shapes and finiteness, and test the denoising algebra with known $(x_{\text{data}}, \varepsilon, s)$.

3.4 Report RBF MMD^2 between generated and validation points, with a bandwidth chosen from real validation data and then frozen. Include a sample-size-matched validation-versus-evaluation-only real comparison (equal numbers of points in each set) as a finite-sample reference. Interpret the scatter and metric together, including whether both moon branches appear. Briefly compare the Gaussian-path flow with the linear flow demo; do not compare their raw training MSE values as if they were on one scale.

Problem 4: Label-conditional diffusion ODE on MNIST

4.1 Load original MNIST with `ToTensor()`, which converts grayscale pixels to floating-point values in $[0, 1]$, and transform each pixel to $[-1, 1]$ by $x_{\text{data}} = 2x - 1$. Keep $(B, 1, 28, 28)$ image tensors and digit labels $c \in \{0, \ldots, 9\}$. Reserve validation rows from the 60,000 official training images and keep the official 10,000-image test split for final evaluation. A stratified subset (preserving per-digit proportions) on the central processing unit (CPU) is allowed if it contains at least 6,000 optimizer-training and 1,000 validation images; report indices or seed.

4.2 Adapt the conditional convolutional neural network (CNN) in the MNIST flow demo so its output is predicted Gaussian noise with the input image's shape. Feed the noised image, time s, and digit c to the network; use (G) and the noise-MSE loss above. Use `nn.Module`, standard convolution layers, autograd, and Adam (adaptive moment estimation, an optimizer). Show tensor-shape and finite-gradient tests. Plot training and fixed-perturbation validation MSE against optimizer updates, and report the best checkpoint (saved model weights) selection rule chosen from validation data only.

4.3 For each requested digit 0–9, generate ten images from independent initial noise with 50 explicit Euler steps in decreasing s, using a fixed seed. Display a 10-by-10 image grid and a five-time-point trajectory grid with one column per requested digit. Convert to $[0, 1]$ only for display and for the evaluation features. State the final-denoise, clipping (truncating values outside the display range), and solver conventions (the numerical time-step rule); record sampling time and network evaluations. Include several imperfect outputs, not just selected successes.

4.4 Evaluate generated images with two complementary checks: (i) label agreement with a *separate* digit probe, a classifier trained on real images to predict digit identity; report that probe's accuracy on untouched real test images before computing the fraction of generated images whose probe-predicted digit matches the requested digit; and (ii) RBF MMD^2 on a fixed 14-by-14 average-pooled pixel representation, obtained by replacing each 2×2 block of the original image with its mean, with a real-versus-real reference. Use at least 500 generated and 500 held-out real images when feasible, with the same count per digit in generated and real groups. Use a second disjoint, class-matched real group (the same per-digit counts) for the real-versus-real reference; report all counts. Explain why probe agreement and pooled-pixel MMD cannot alone establish human recognizability, full distributional coverage, or a conventional Fréchet Inception Distance (FID) score, which compares means and covariances of features from a pretrained Inception network. A documented fixed-feature Fréchet analysis (the same Gaussian-feature comparison in a declared feature space) or Kernel Inception Distance (KID, an MMD^2 on Inception features, usually with a polynomial kernel) may supplement these checks but does not replace the visual grids.